

Scaling laws — small $\rightarrow$ large scale $\Rightarrow$ hyperparameter transfer $\rightarrow$ problem: ~~overfit~~ regularization

hardware + size of data $\xrightarrow{\text{limit}}$ parameters

data curation $\leftarrow$ recipe

proximity
of data type

↑
annoted + arguementation + clean + selection

perplexity

uncertainty of evaluation

complexity

uncert

utilization

Tensor core
(N-GPU)
FP16

operation

matrix

compute

uncertainty

utilization

Tensor core
(N-Gym)
FP16 $\rightarrow$ FP32

matrix systolic array

VS
ALU

operation
(+) (x)

compute

memory

data size
middle

doct

storage

cpu

hardware

cpu

data type

element + type

memory

Byte

8bits

8bits

flop

flop/s (FLOPS)

FP16

FP32

bfloat16

sign

double

64

PyTorch

parameters / activation

memory-bandwidth.

tokenizer

e.g NLP

characters $\rightarrow$ bytes $\rightarrow$ words $\rightarrow$ byte pair encoding = BPE
 large sequence $\uparrow$ large unknown
 e.g. [5] + [4] $\rightarrow$ strange
 softmax
 about relative

batch.
↓
B L D H

A handwritten diagram illustrating the internal structure of a Transformer block. The diagram shows the flow of data through various layers and operations:

- Input:** Represented by a small grid icon, with an arrow pointing to the **dimension** label.
- Layer:** The input enters a box labeled **layer**, which contains **dropout** and **weight decay**.
- Position:** An arrow labeled **position** points to the **Transform** block.
- Transform:** A central box labeled **Transform** receives input from the **layer** and the **position** information. It outputs to the **attention** mechanism.
- Attention:** Below the **Transform** box, the terms **Q** and **K** are written, followed by **Flatten** and **RM's norm**.
- Hyperparameters:** A label **Hyperparameters** points to the **layer** and **Transform** components.
- Batch Norm:** An arrow labeled **batch norm** points to the **layer** box.
- Memory:** A circle labeled **Memory** is connected to the **layer** box.
- Layer Norm:** The output of the **layer** box goes through **Layer Norm**, which is split into **pre** and **post** components.
- Output:** The final output is labeled **GOAL MSA** (Multi-Head Self-Attention) and **Soft upper**.
- GPU:** A circle labeled **GPU** is connected to the **Layer Norm** output.

optimizer

- Adam
- SGD
- Momentum

Memory ---

parameters
format
vs
layer
batch norm
pre
post
stability
forward network

(FFN) Feed forward network
↑ ++ ↓
activation function
(sparse) ReLU → GELU
(BERT)
SwiGLU ←
(Gated ++).
parameters

swiglu $\leftarrow$
(Gated +).
parameterst

(work)

* think from first principle

* tiling

* square material e.g. 16×16 , 32×32

* CUDA model

multiplication of matrix.

GPU structure - TPV

Scalar unit if / else x overhead instead of min/max offset x

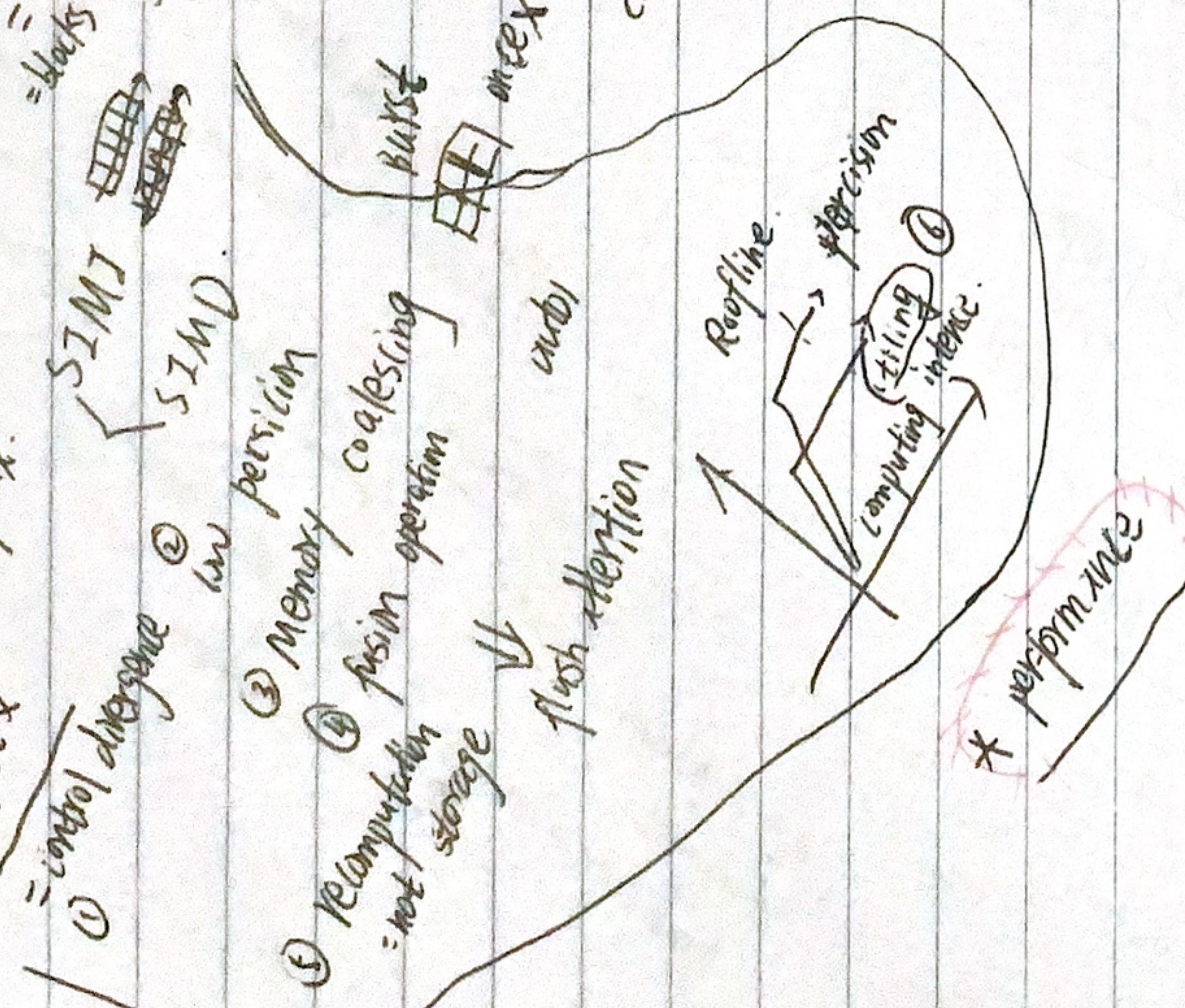
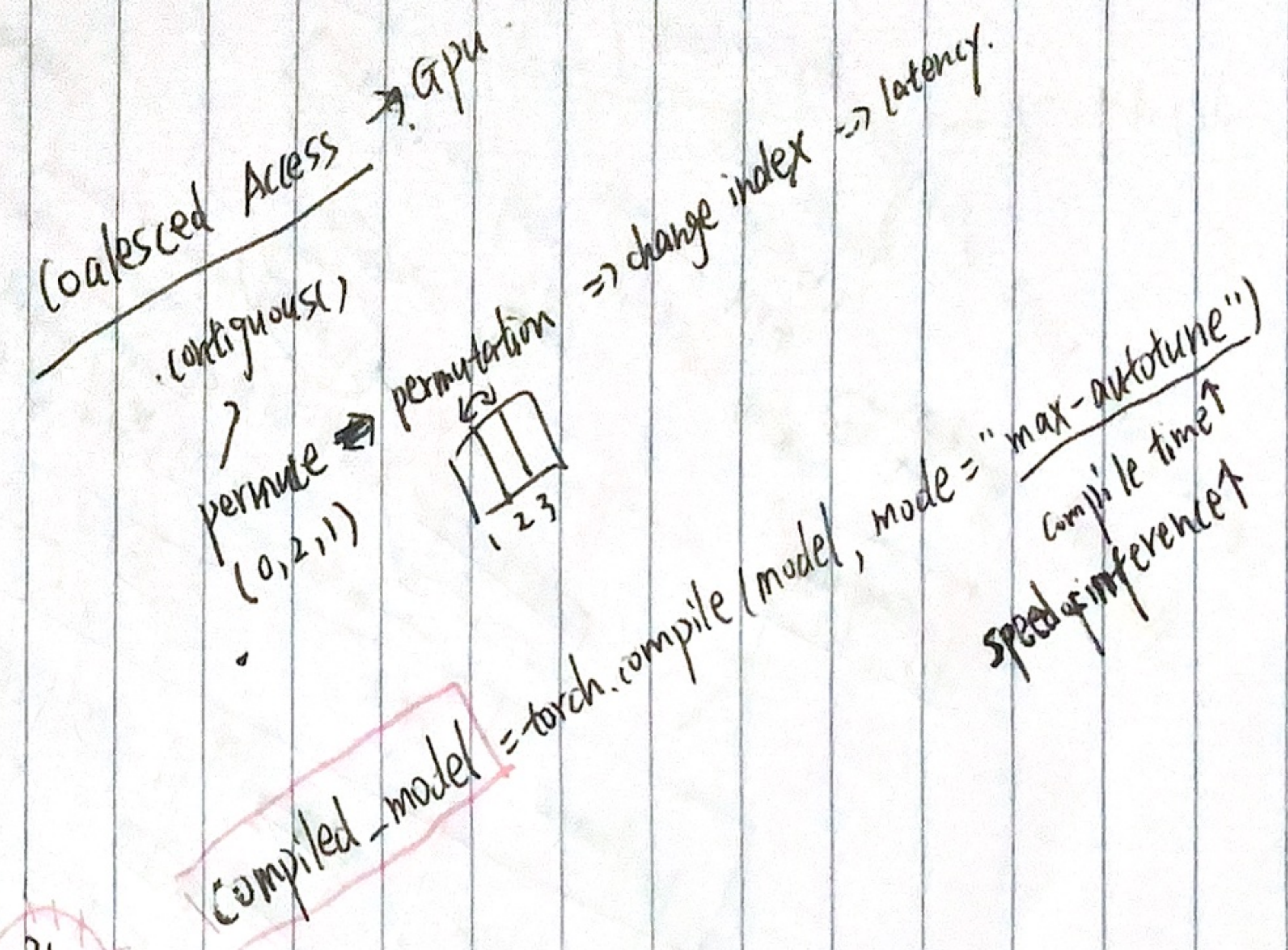
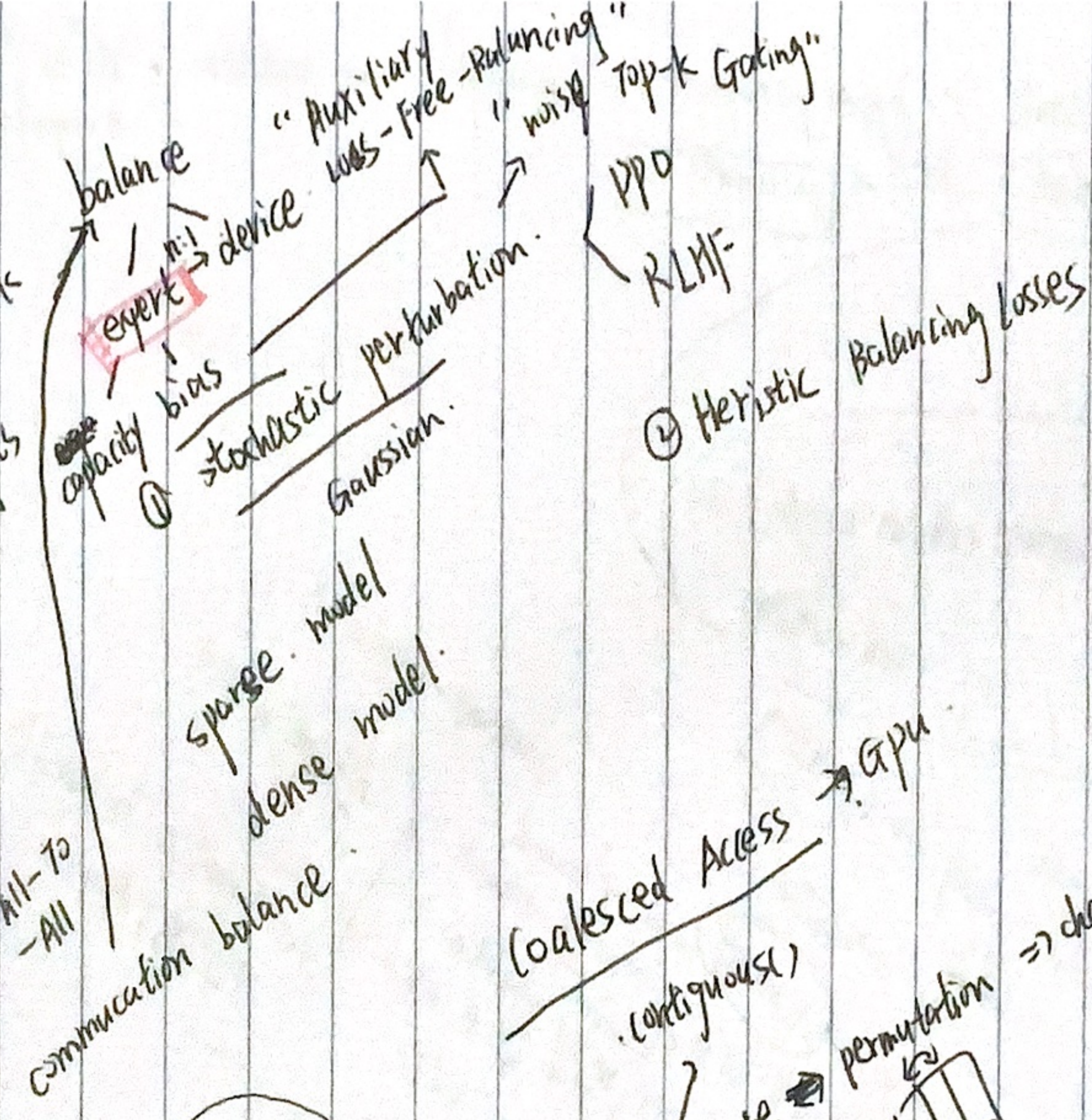
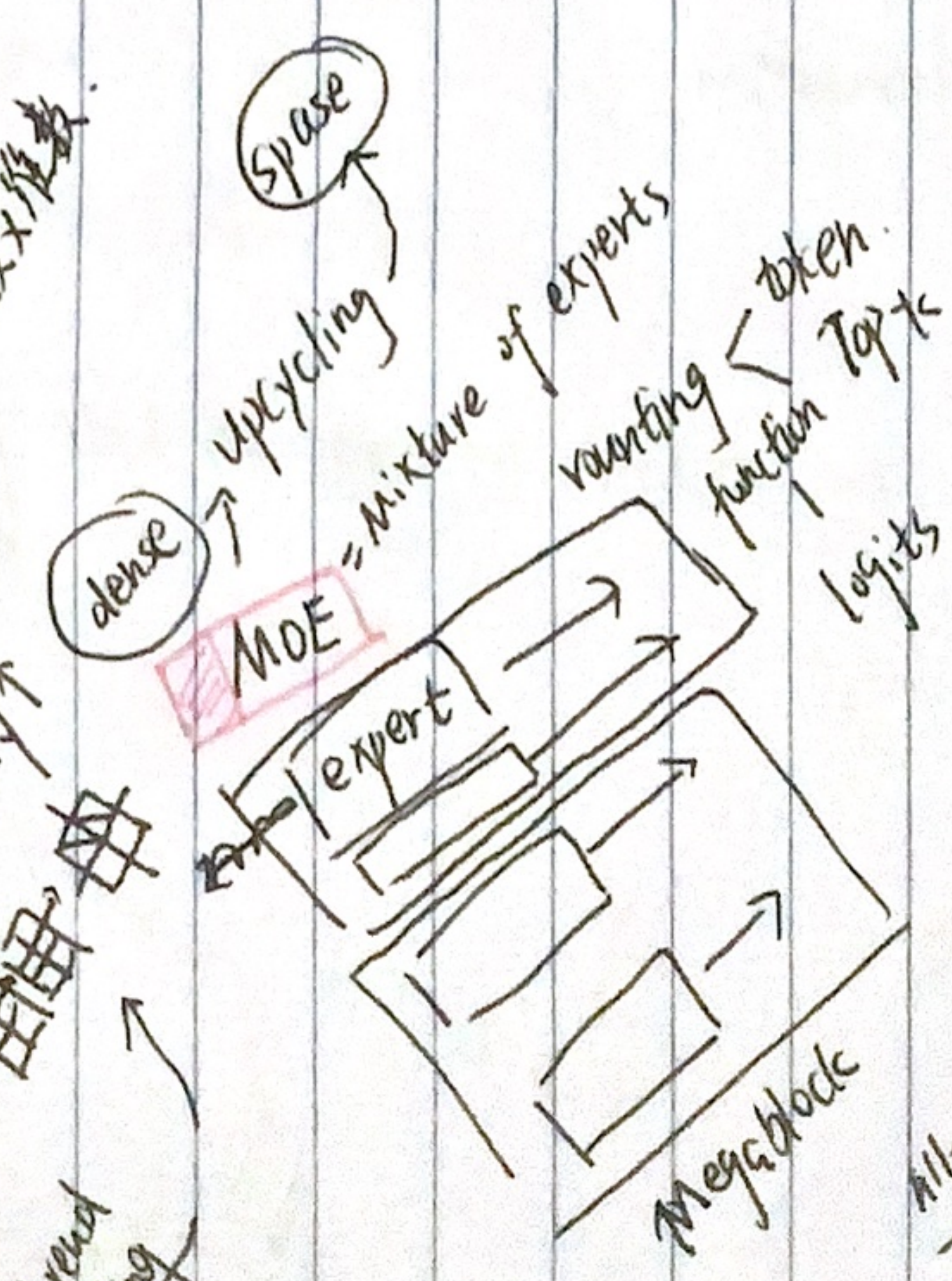
access of memory ↑

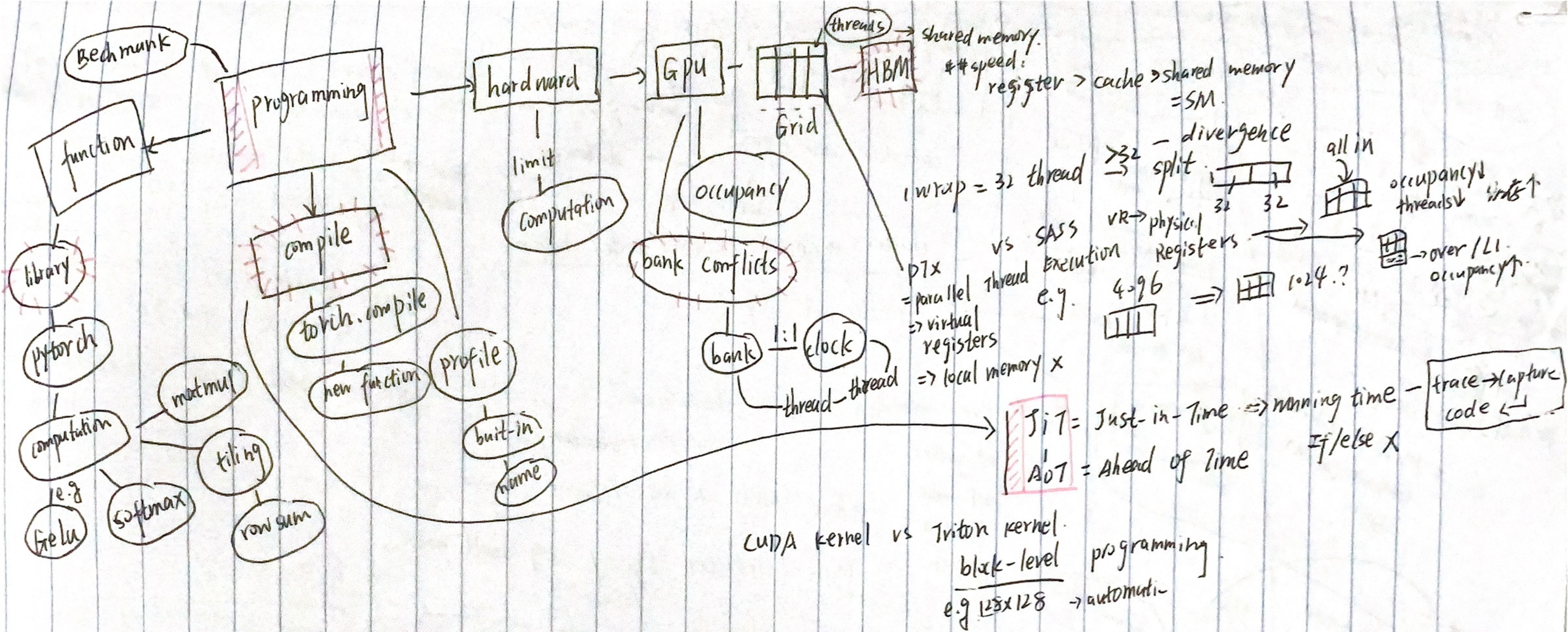
dense

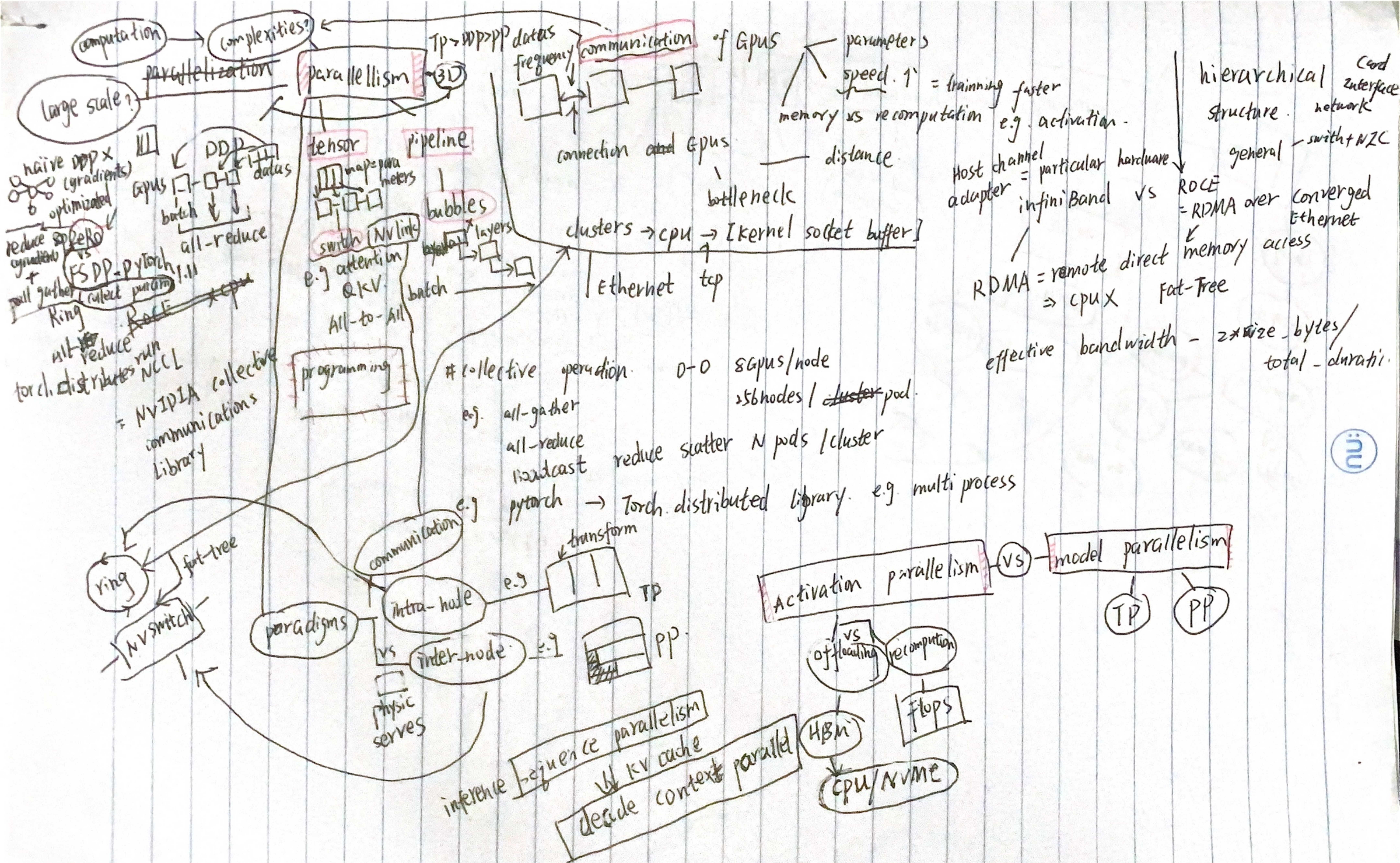
e.g. attention k_i cache
整型 × 序列长度 × 头数 × 维数

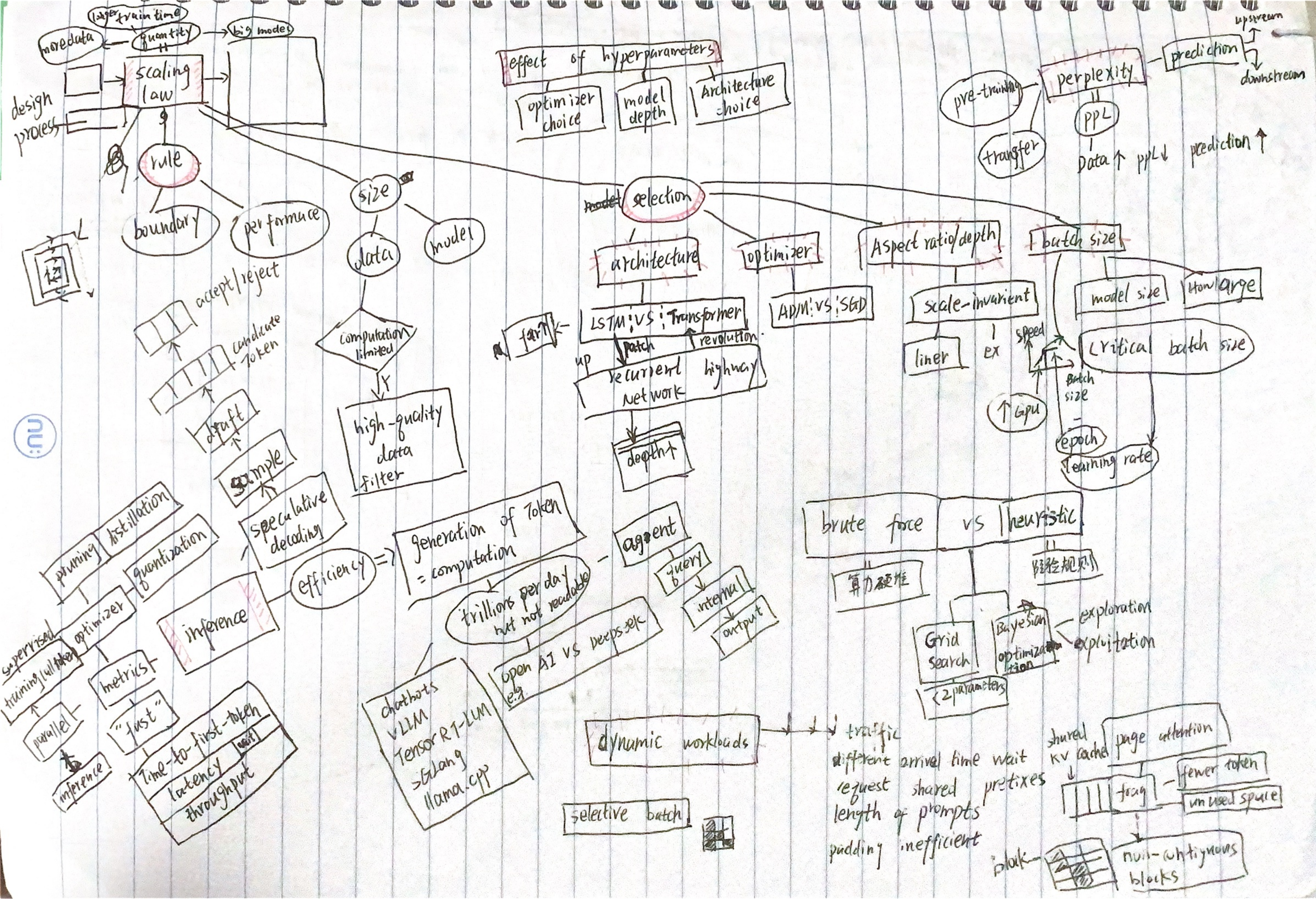
HD - 带宽 - 延迟

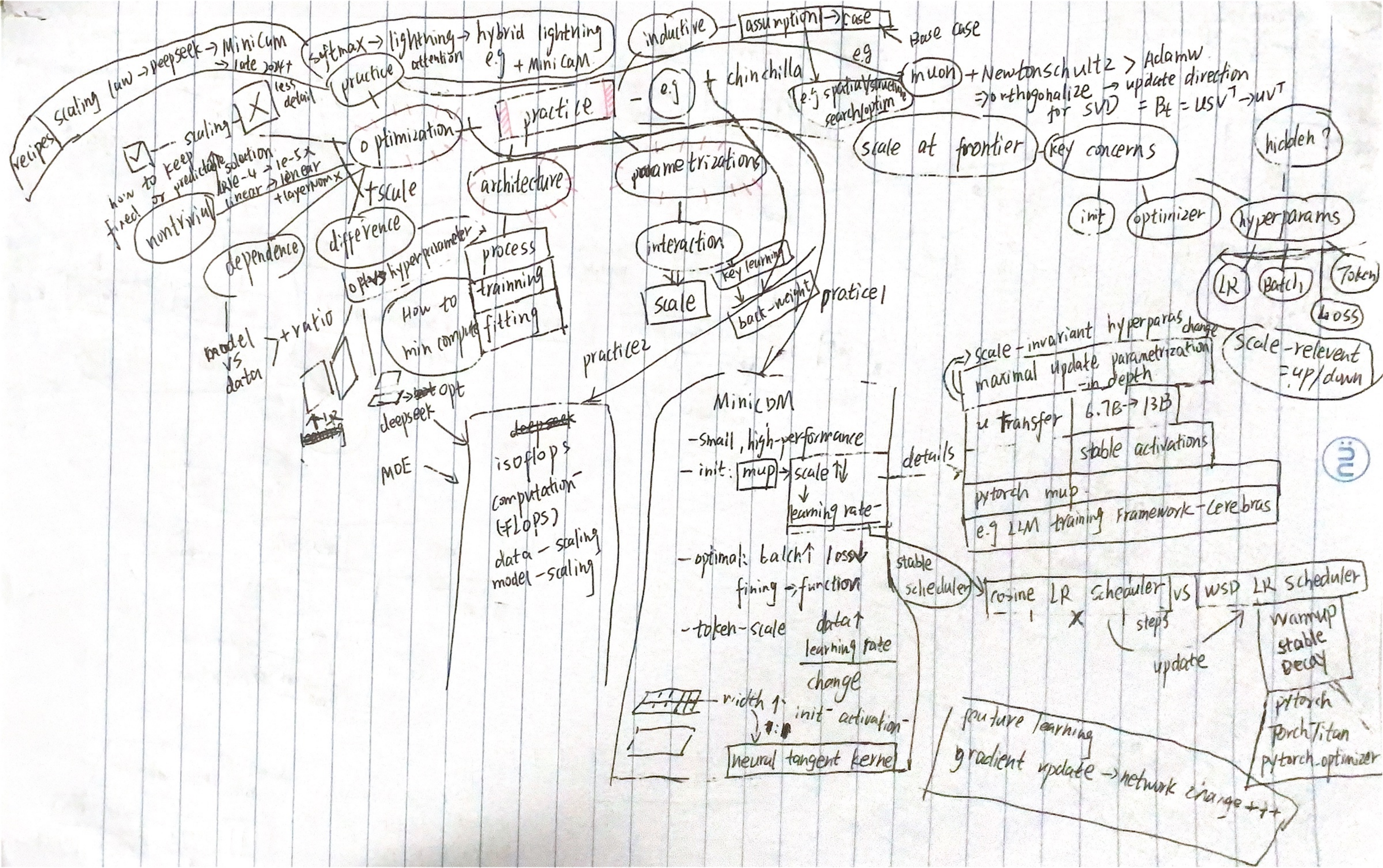
40M

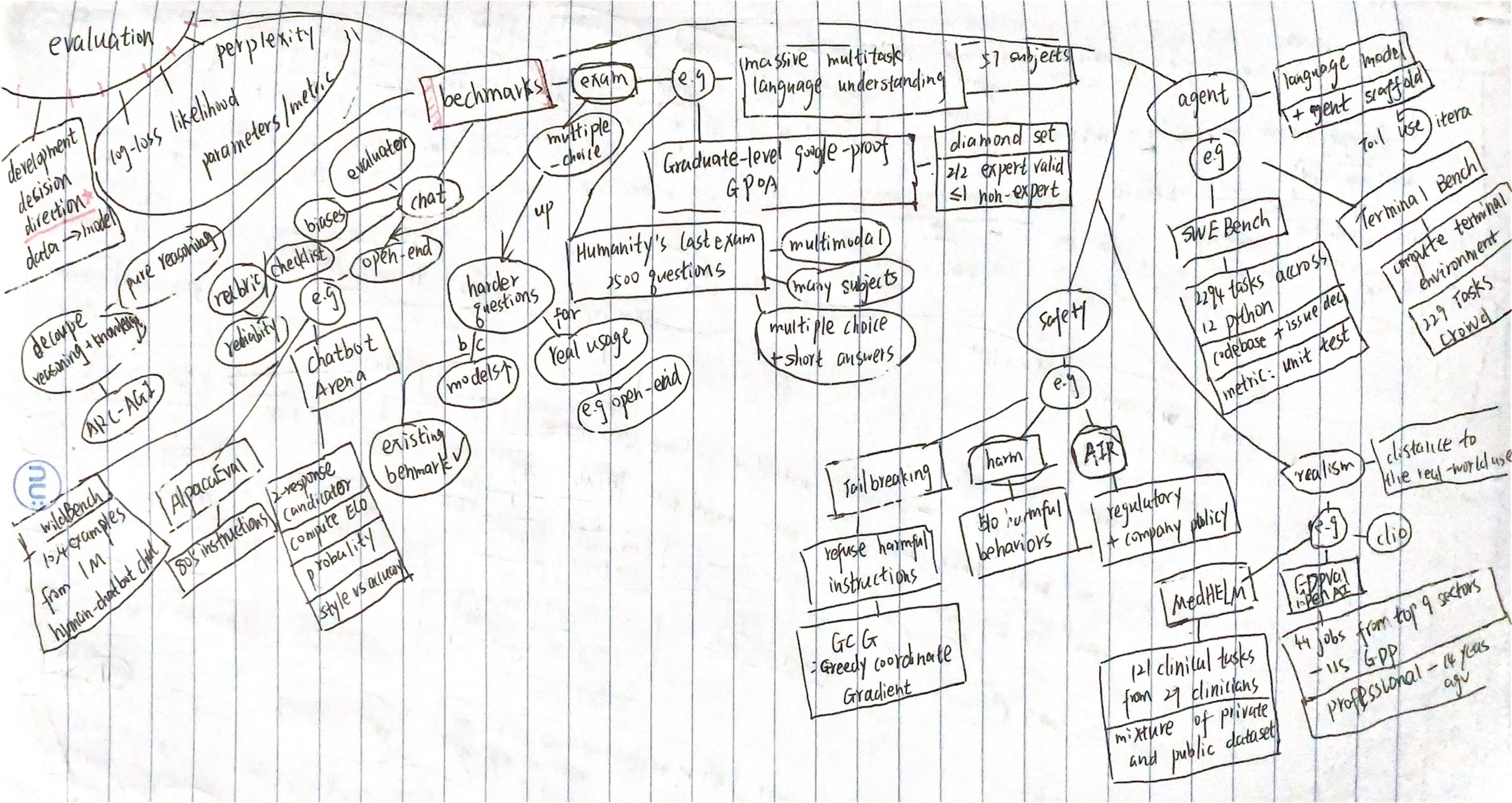


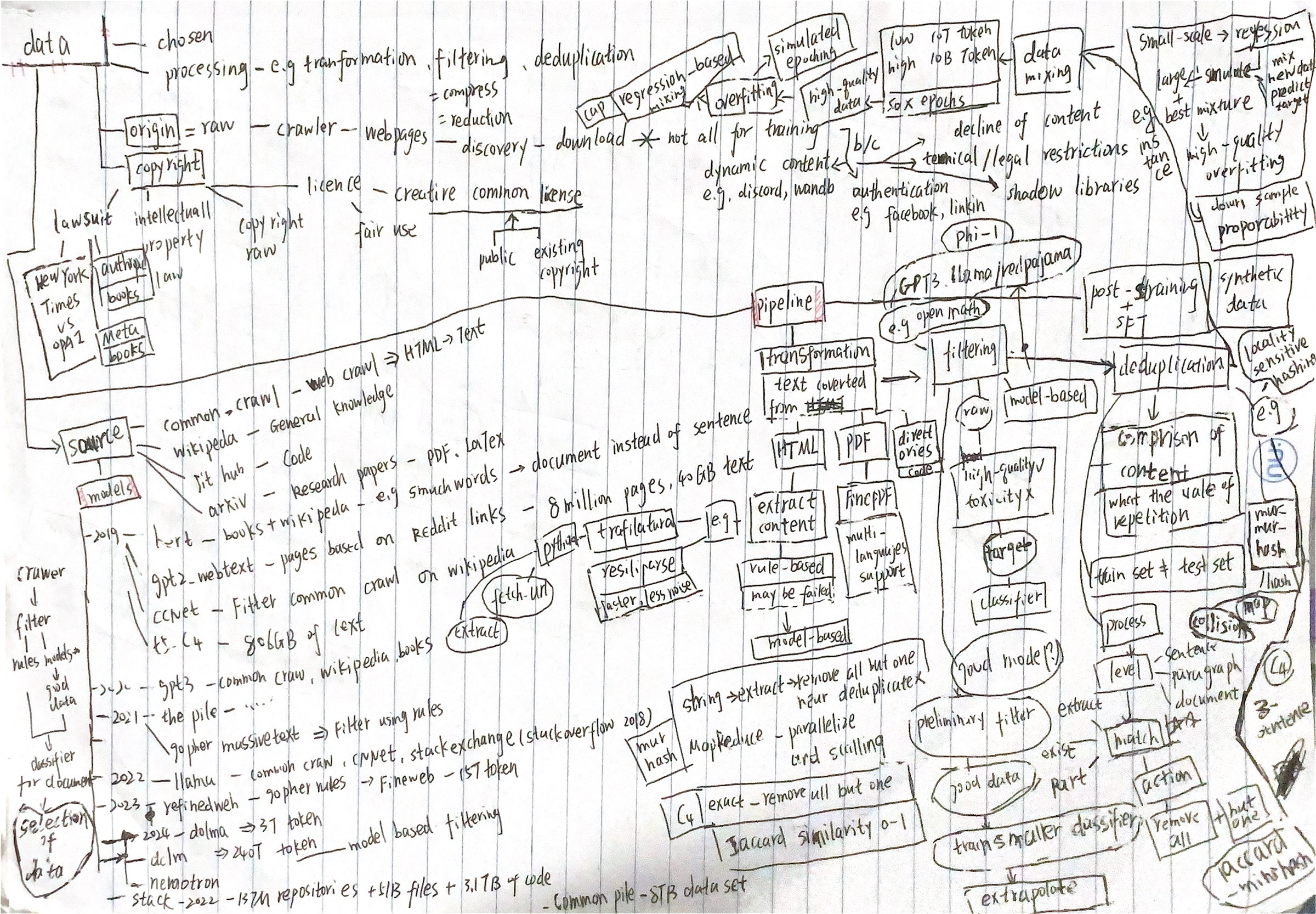






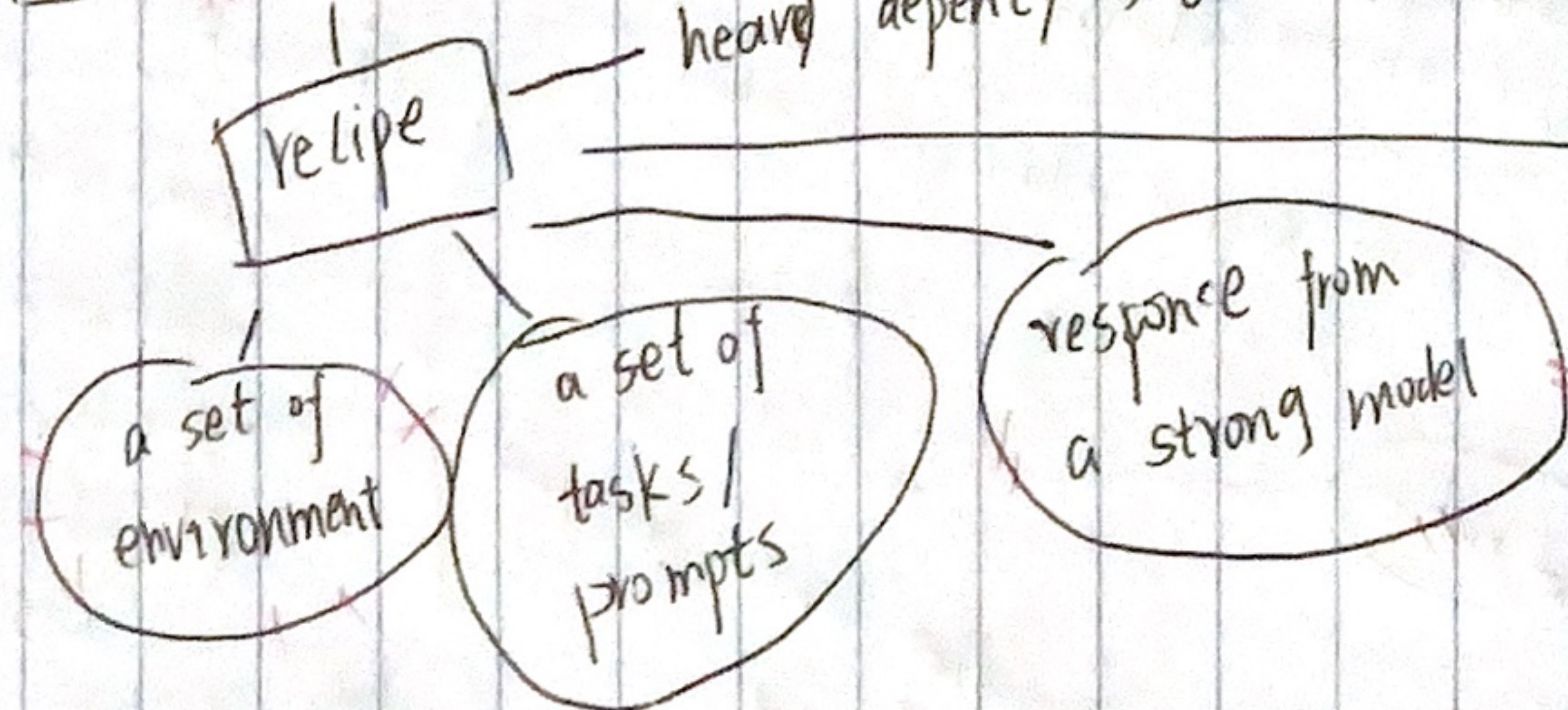






post-training — task-dependent

heavy dependency → docker images x → good models scope tasks without execution feedback



e.g. open thoughts

- science
- code
- math
- chain of Thought = multi-step Reasoning ⇒ sequence length ↑

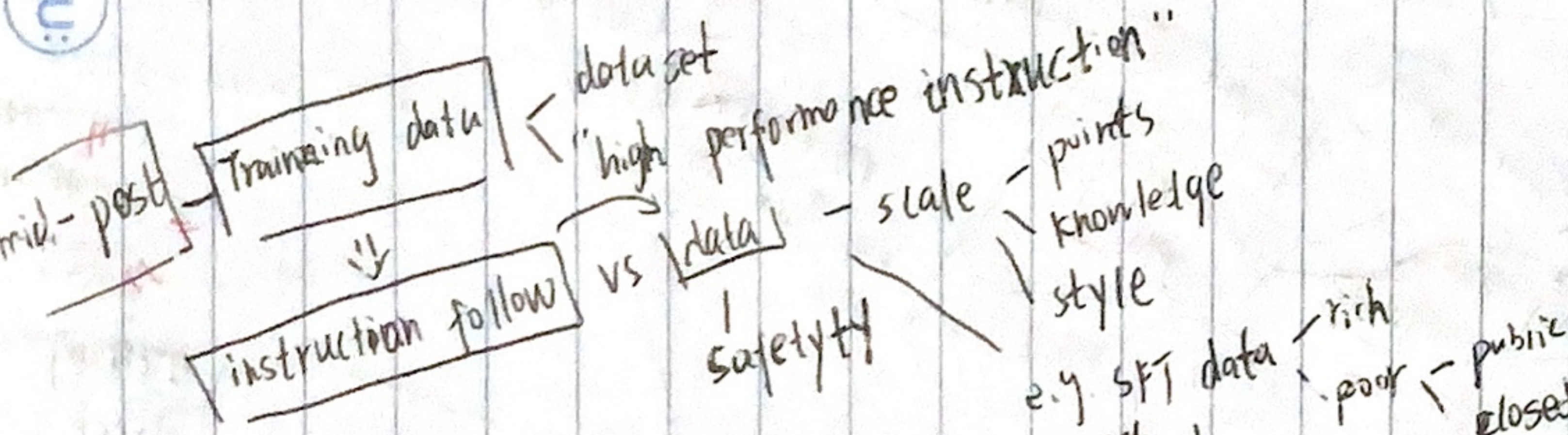
SWE-smith-2025
SWE-zero-2026
SWE-rebench
SWE-12M-trajectories

scale SWE-zero up to 12M agent trajectories

lightweight

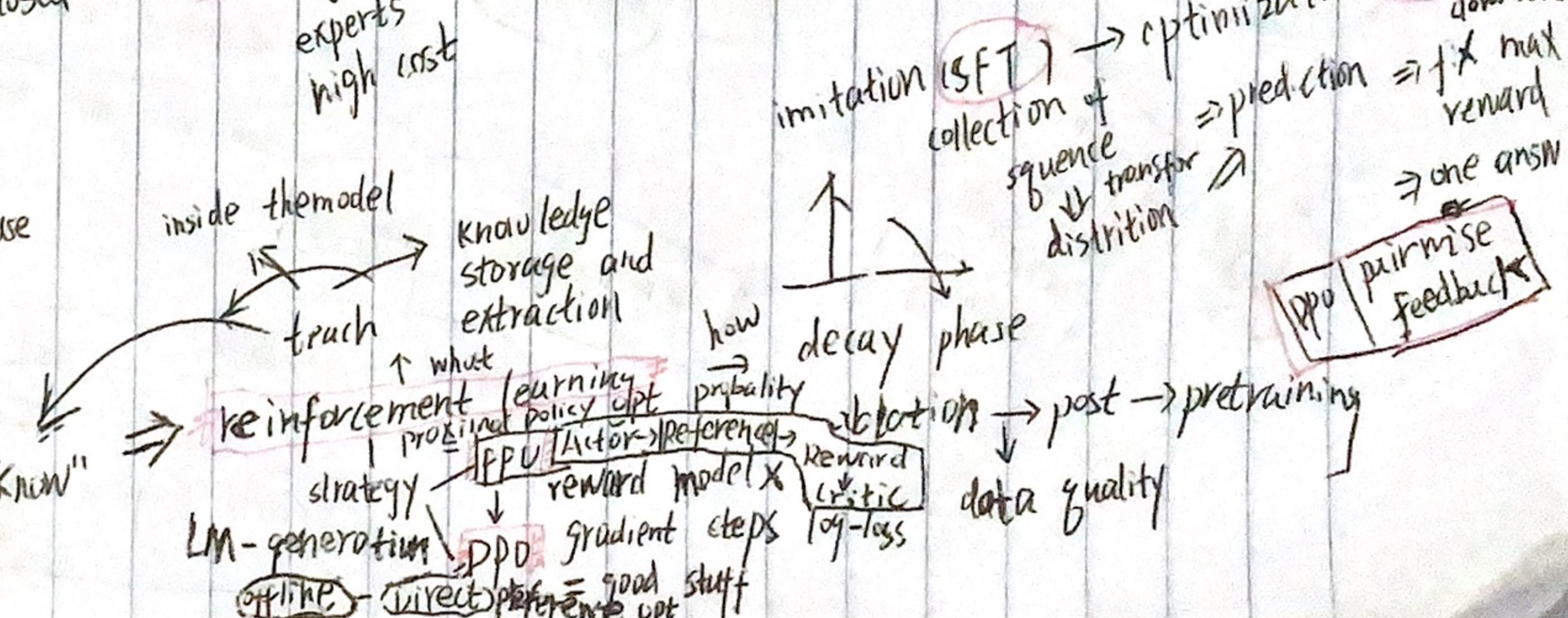
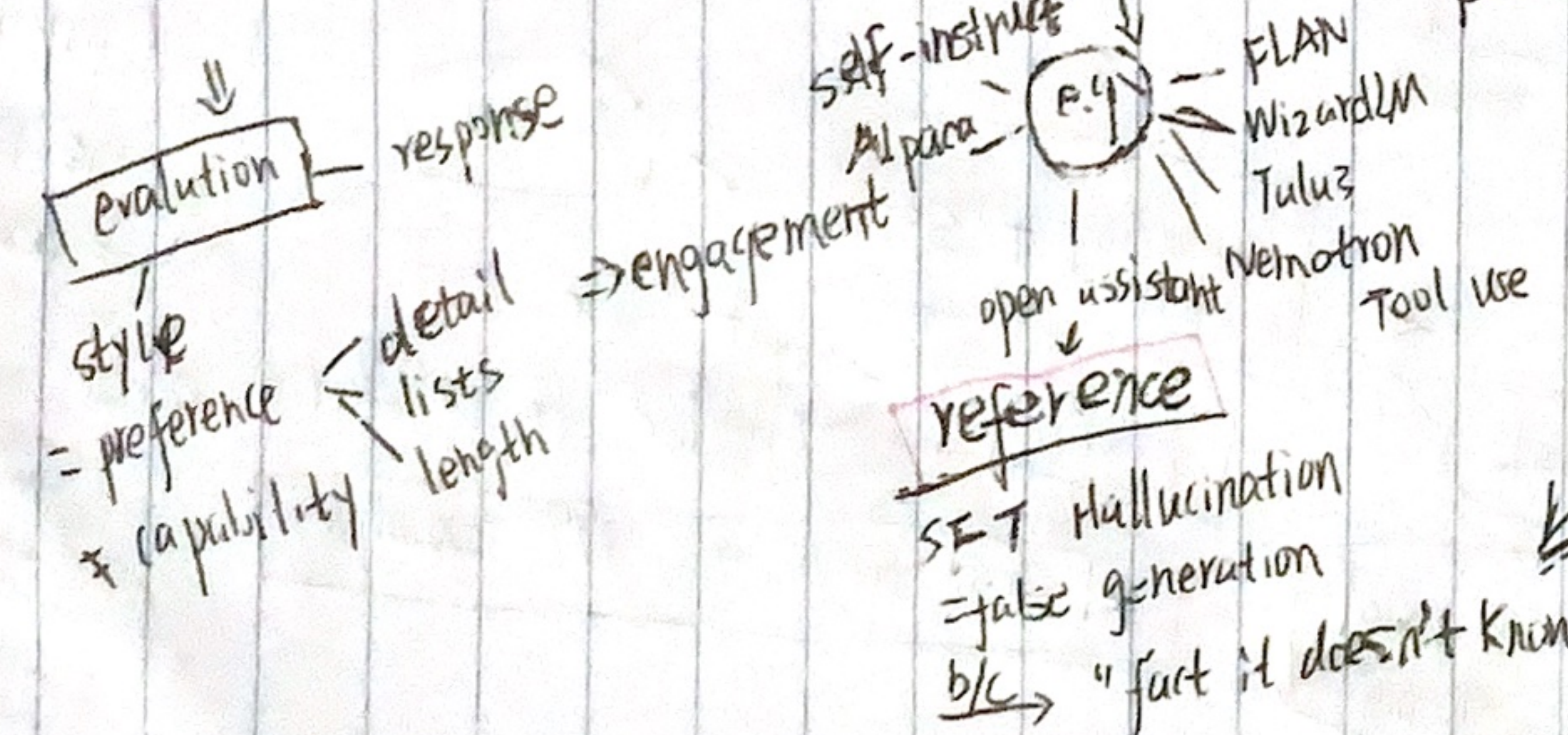
used SWE-rebench-v2 tasks 32k executable tasks + 120k non-executable

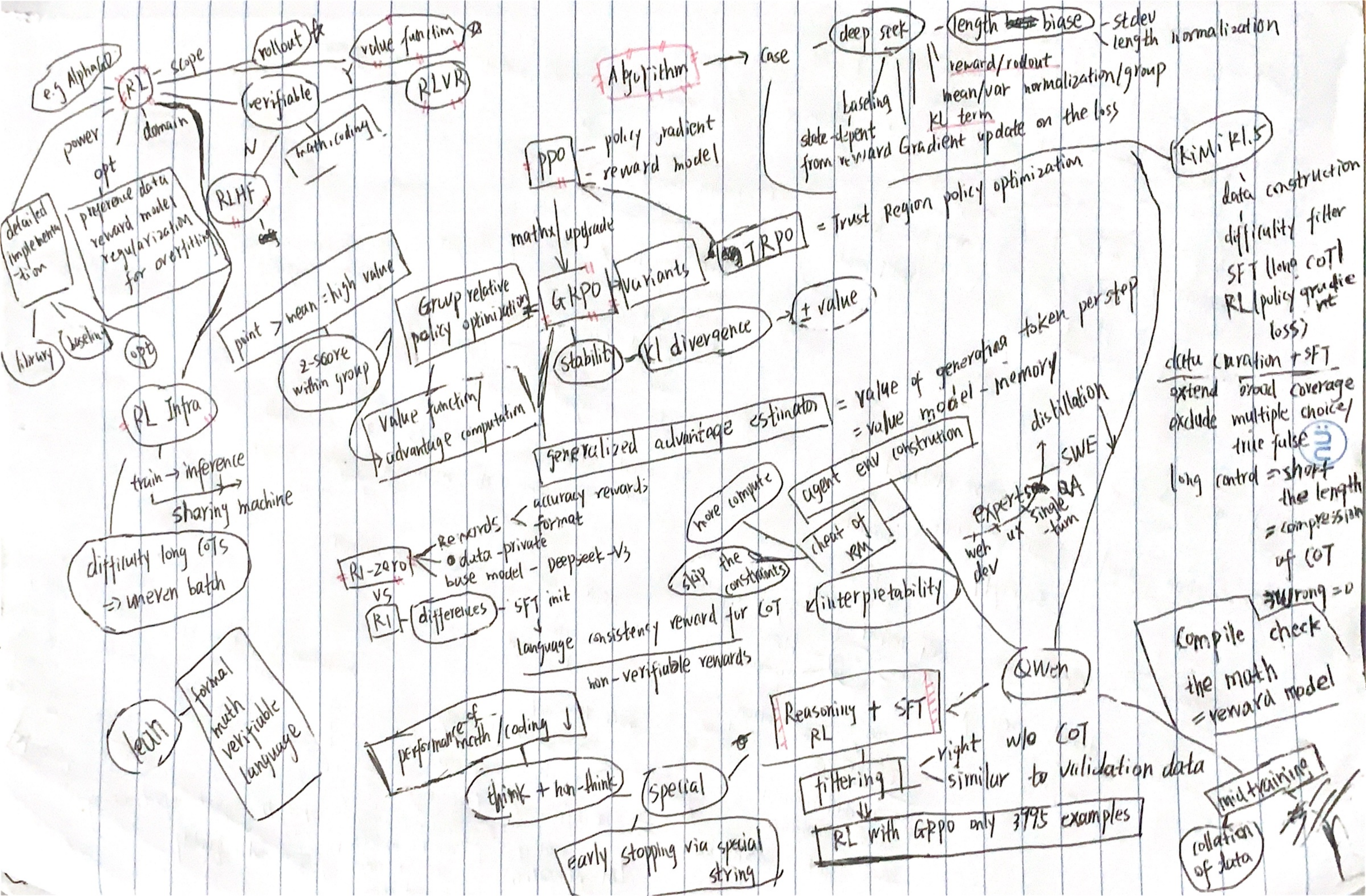
ran mini-coder-1.7b, mini-swe-agent scaffold
Agent-Computer-Interface

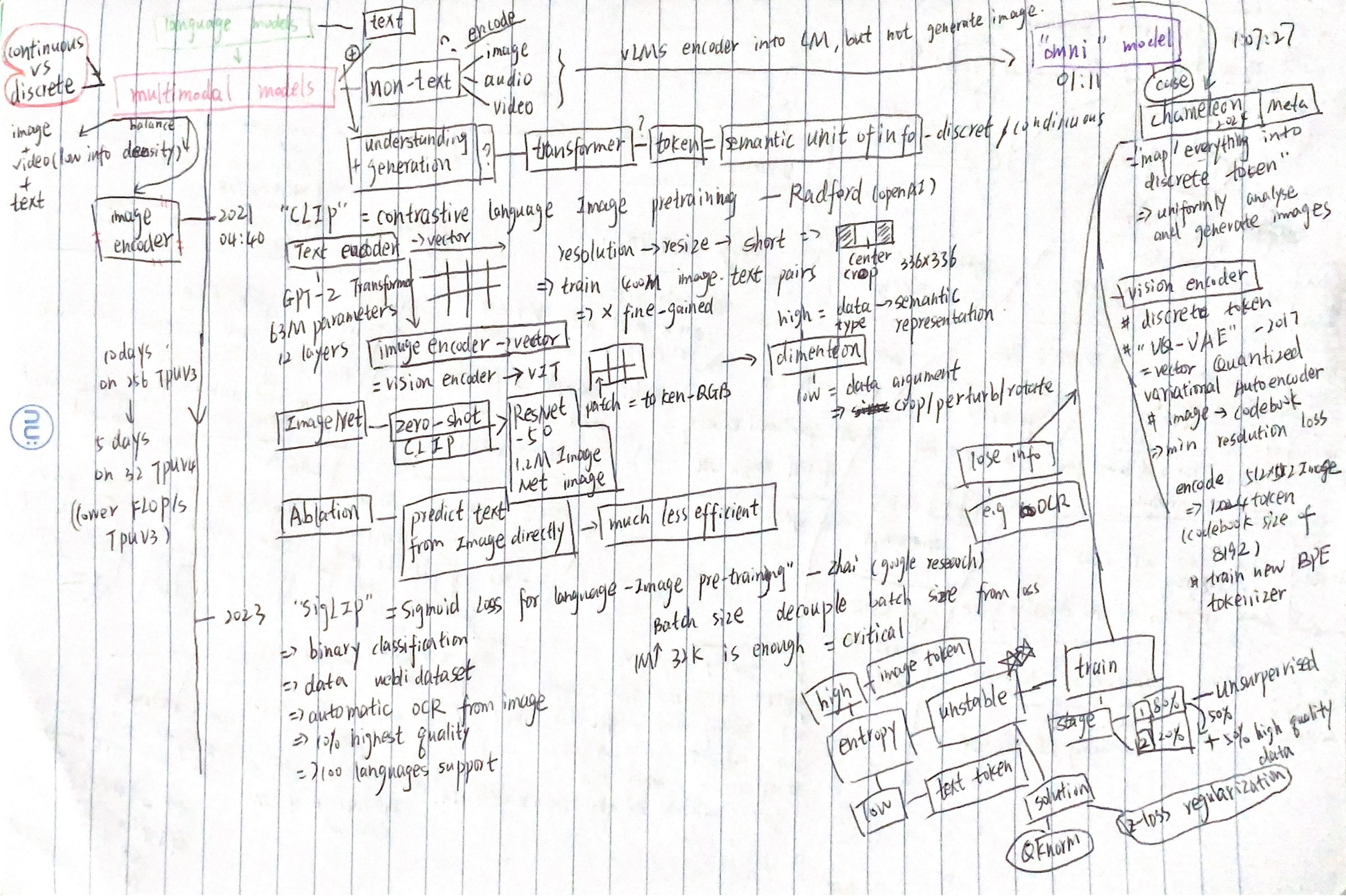


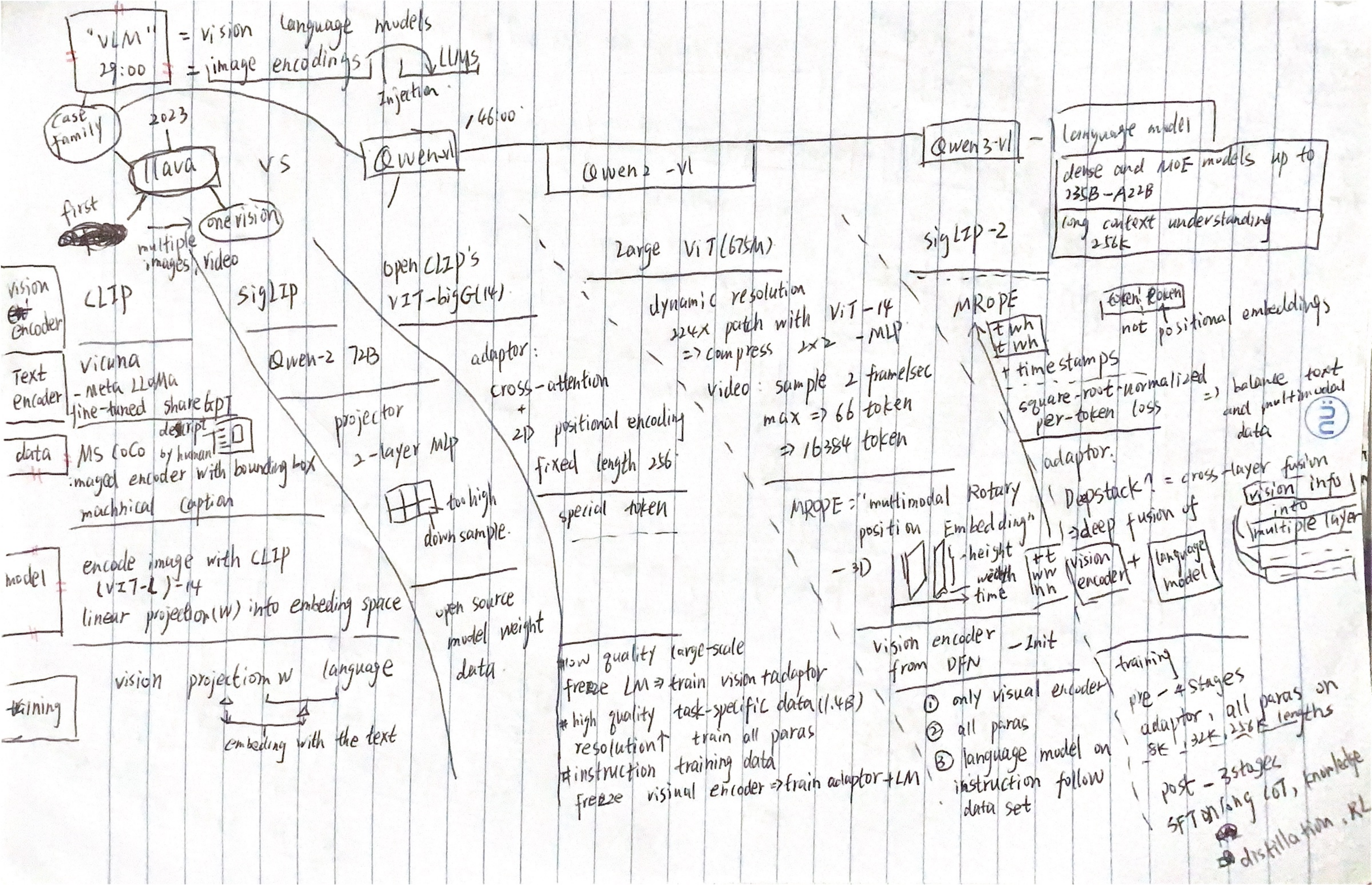
model-based vs domain-driven

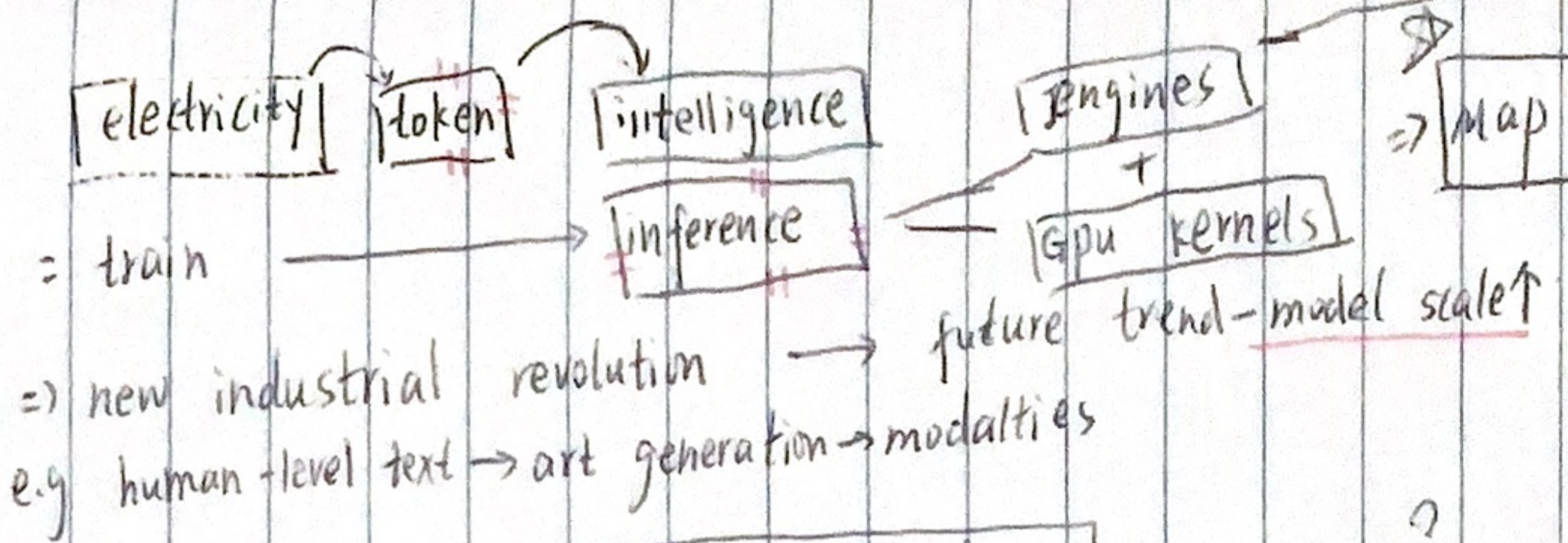
experts high cost











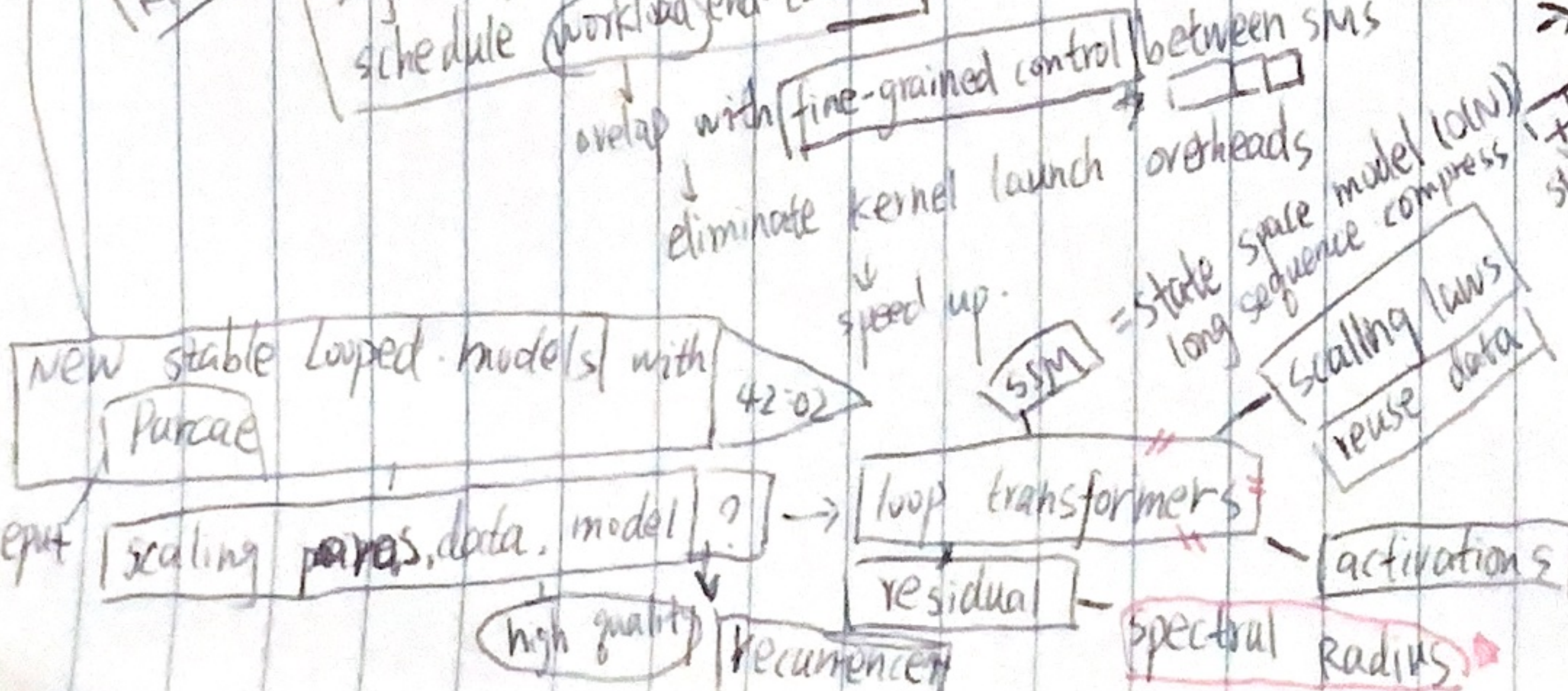
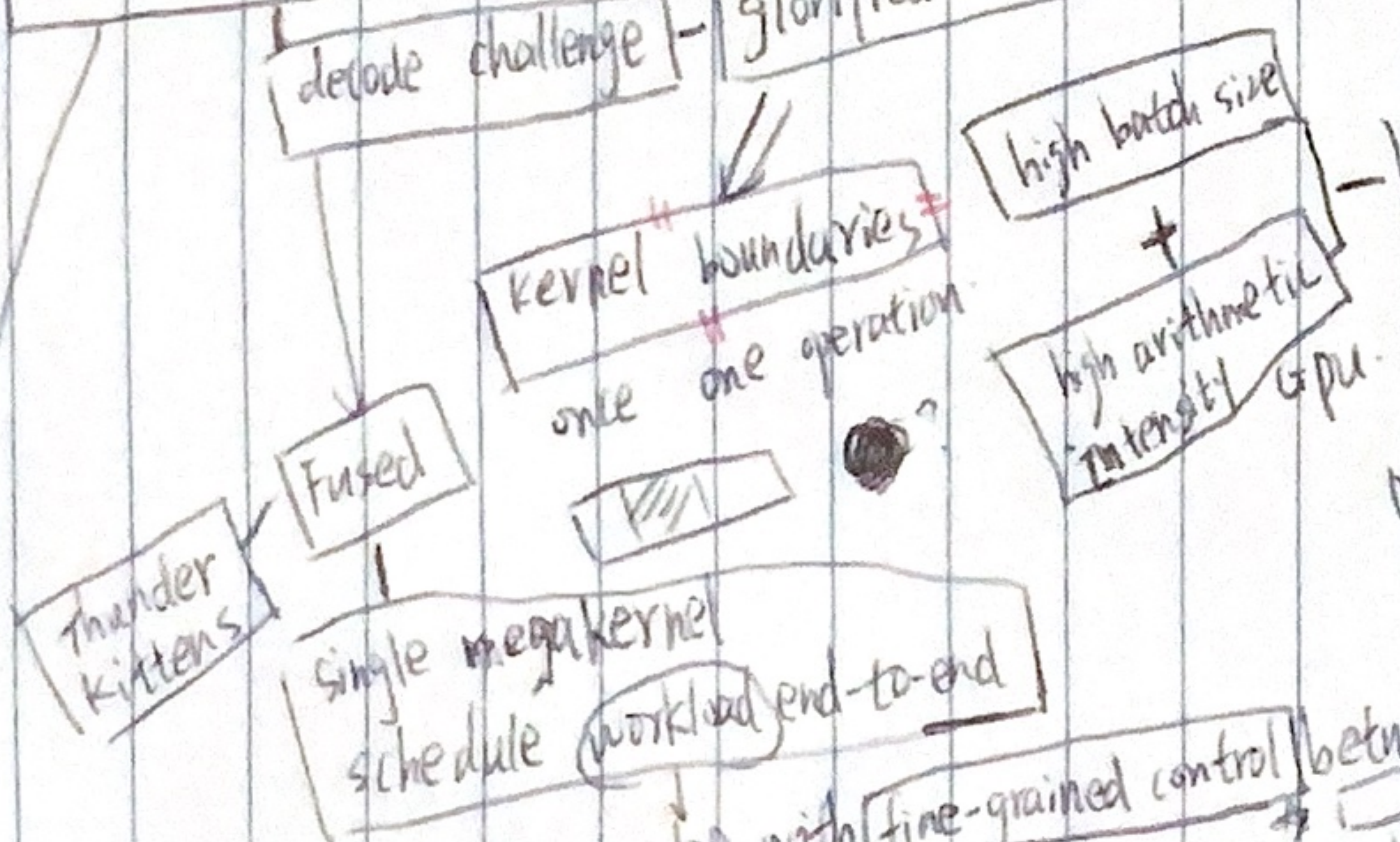
full-stack innovation in ML algorithms

Fast LLM Decode with Megakernel

34:38 = parallelism X

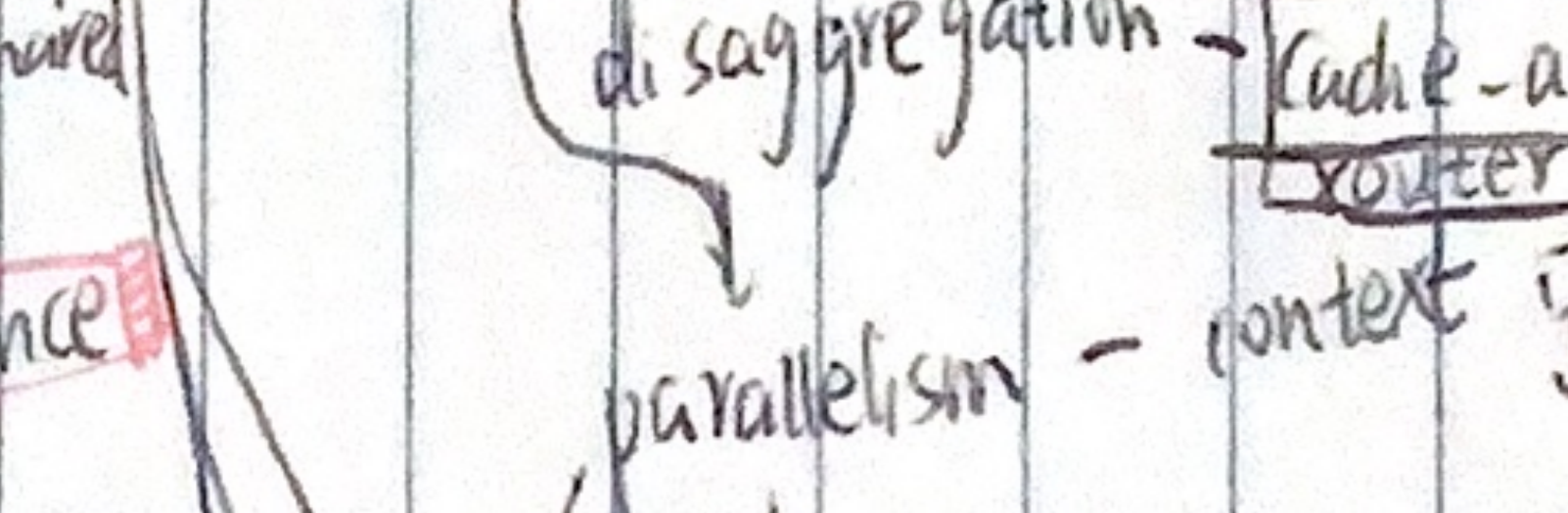
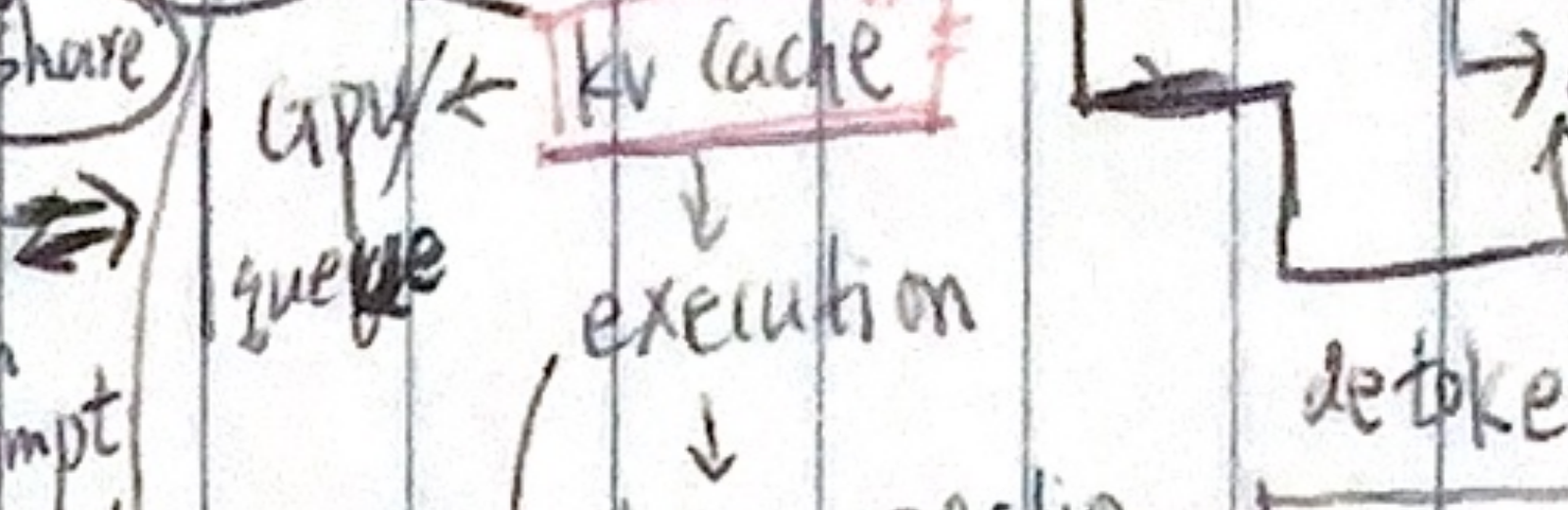
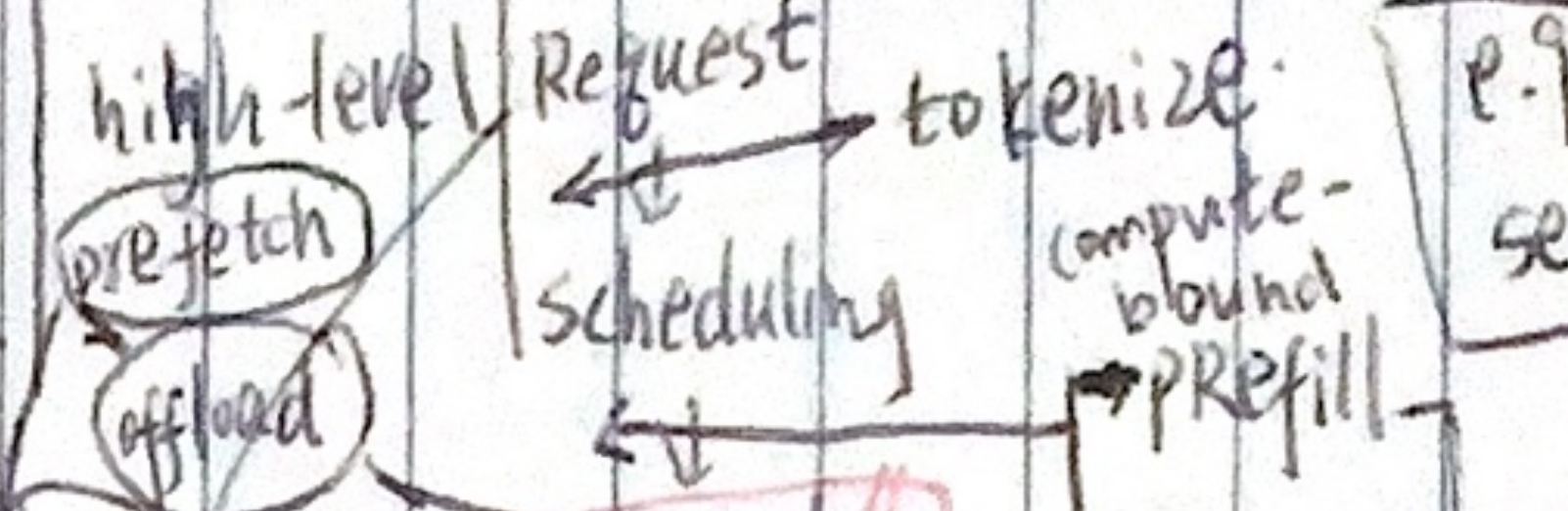
once one token generation

glorified memory loader



Map ML operations DAGs?

lifetime of a token



cache hit rate

input/output

workload

e.g. interactive session

decode loop

memory-bound

one token/step

detokenizer

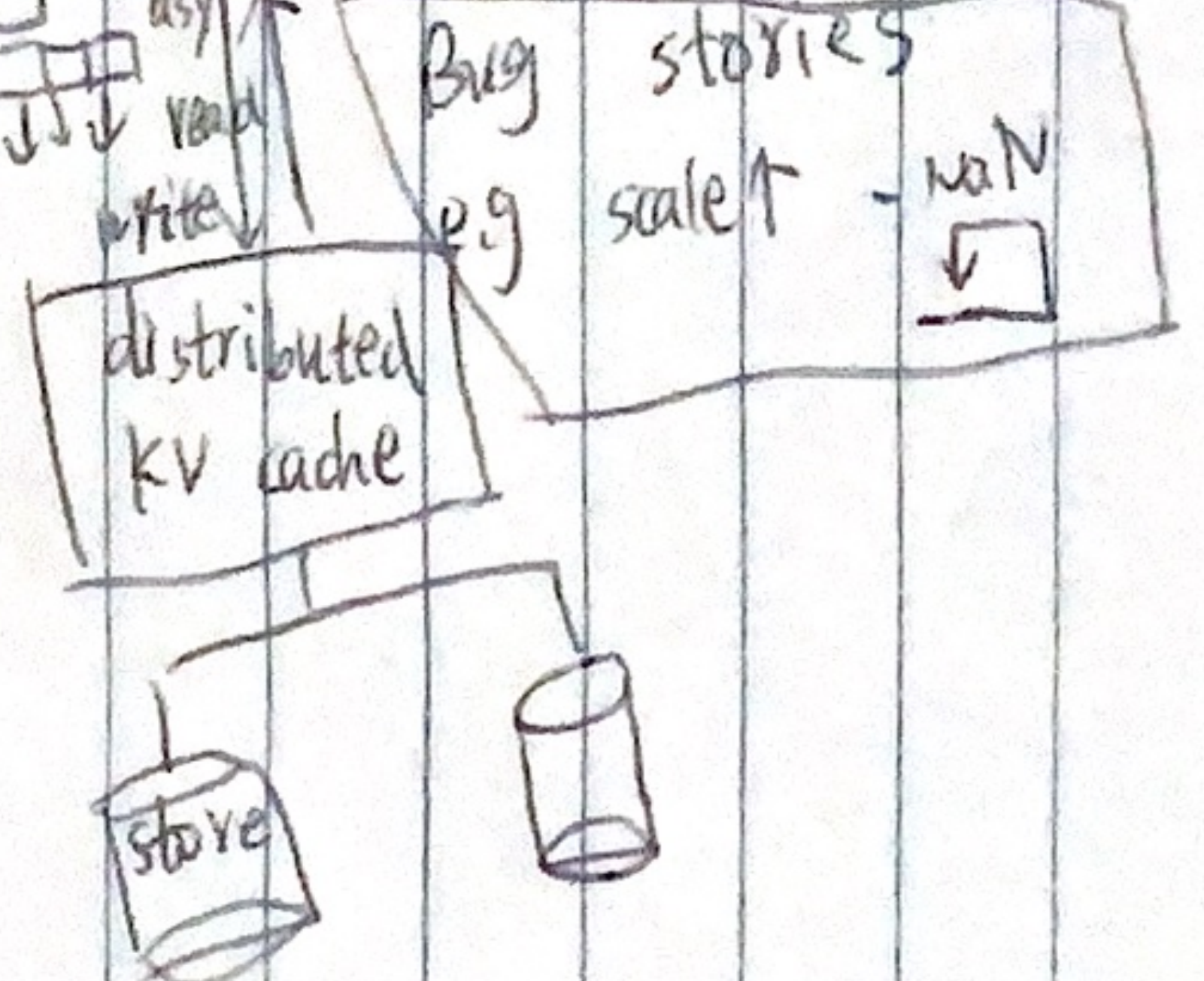
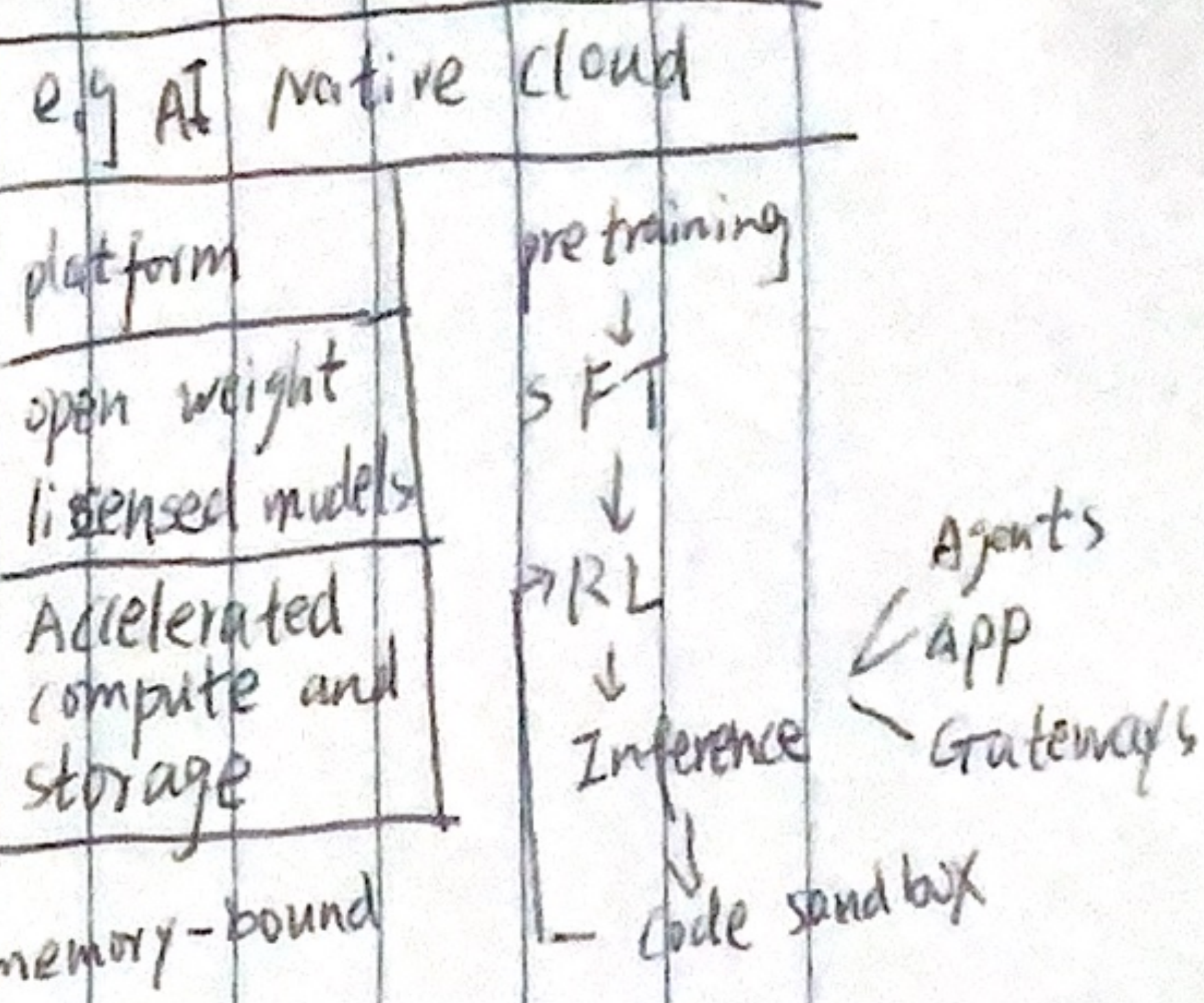
cache-aware router

pre-prefill - cold requests low cache hit

warm requests

prefill

decode (QD)



CS 336

—

Stanford

Language Model from Scratch